SIMATS ENGINEERING — ASSESSMENT TOOL 1

Detailed Solutions — Case Study (Annexure A)

Course Name: Artificial Intelligence **Course Code:** CSA17

CO3: Knowledge Representation & Reasoning (Propositional Logic, FOL, Inference, Resolution, Chaining) **Total Marks:** 40

Case Study 1 — Smart Medical Diagnosis System (10 Marks)

(a) Propositional Logic Representation

Define the atomic propositions used to encode the knowledge base:

Symbol	Meaning
F	Patient has Fever
C	Patient has Cough
R	Patient has Rash
B	Patient has Breathlessness
Flu	Possible Flu
Mea	Possible Measles
Pneu	Possible Pneumonia

Using these symbols, the knowledge base rules and facts are represented as:

```
R1: (F ∧ C) → Flu
R2: (F ∧ R) → Mea
R3: (C ∧ B) → Pneu
Facts (Patient A): F = True, C = True, B = True
```

Each rule is a Horn clause (a conjunction of positive literals implying a single positive literal), which makes the KB directly suitable for forward- and backward-chaining inference.

(b) Forward Chaining — Step-by-Step Trace

Forward chaining is data-driven: we start from the known facts {F, C, B} and repeatedly fire any rule whose full premise is already satisfied, until no new fact can be added.

Step	Rule Checked	Premise Satisfied?	Fact Derived
1	R1: F ∧ C → Flu	F=T, C=T → Yes	Flu = True
2	R2: F ∧ R → Mea	R is unknown → No	— (rule does not fire)
3	R3: C ∧ B → Pneu	C=T, B=T → Yes	Pneu = True

Result: No further rule can fire (R2 remains blocked since Rash is never observed). The fixed point of forward chaining yields the final derived diagnosis set:

```
Derived = { Flu = True, Pneu = True }
```

Conclusion: Patient A is diagnosed with **Possible Flu** and **Possible Pneumonia**; Measles is not indicated because Rash was never asserted as true.

(c) Backward Chaining — Goal: 'Patient A has Pneumonia' (Pneu)

Backward chaining is goal-driven: we start from the goal Pneu and recursively reduce it to sub-goals using the rule whose conclusion matches the current goal, until every sub-goal is confirmed as a known fact.

```
Goal: Pneu
Pneu matches R3: C ∧ B → Pneu, so reduce Pneu to sub-goals {C, B}
Sub-goal C (Cough): C ∈ Facts → TRUE
Sub-goal B (Breathlessness): B ∈ Facts → TRUE
Both sub-goals of R3 proven → Pneu is PROVEN TRUE
```

Goal-reduction tree:

Level	Node
0 (Goal)	Pneu
1 (via R3)	C AND B
2 (leaves)	C = Fact(True) B = Fact(True)

Since both leaf nodes (C and B) resolve to known facts, the goal 'Patient A has Pneumonia' is formally **VERIFIED TRUE** by backward chaining — consistent with the forward-chaining result obtained in part (b).

Case Study 2 — Autonomous Traffic Management Agent (10 Marks)

(a) Unification of Rule 1 with the Given Facts

Rule 1 states: ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x). We unify its premises against the facts Vehicle(Ambulance) and EmergencyType(Ambulance).

Step	Unification Call	Resulting Substitution θ
1	UNIFY(Vehicle(x), Vehicle(Ambulance))	θ = { x / Ambulance }
2	UNIFY(EmergencyType(x), EmergencyType(Ambulance)) under θ	θ = { x / Ambulance } (consistent, unchanged)

Both premise literals unify consistently under the single substitution, so the **Most General Unifier (MGU)** is:

```
θ = { x / Ambulance }
```

Applying θ to Rule 1 grounds it to: Vehicle(Ambulance) ∧ EmergencyType(Ambulance) → ClearPath(Ambulance). Since both antecedents are known facts, Modus Ponens immediately yields the new fact **ClearPath(Ambulance)**.

(b) Resolution Proof that Proceed(CarA) is True

Step 1 — Convert every implication to CNF (P → Q ≡ ¬P ∨ Q; a conjunctive premise A∧B→Q becomes ¬A ∨ ¬B ∨ Q):

Clause	CNF Form	Source
C1	¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x)	Rule 1
C2	¬ClearPath(y) ∨ GreenSignal(y)	Rule 2 (conjunct 1)
C3	¬ClearPath(y) ∨ ¬RedSignal(y)	Rule 2 (conjunct 2)
C4	¬Vehicle(x) ∨ ¬Behind(x,y) ∨ ¬GreenSignal(y) ∨ Proceed(x)	Rule 3
C5	Vehicle(Ambulance)	Fact
C6	EmergencyType(Ambulance)	Fact
C7	Vehicle(CarA)	Fact
C8	Behind(CarA, Ambulance)	Fact
C9	¬Proceed(CarA)	Negated Goal

Step 2 — Resolve to the empty clause:

#	Resolve	Unifier	Resolvent
R1	C1, C5	{x/Ambulance}	¬EmergencyType(Ambulance) ∨ ClearPath(Ambulance)
R2	R1, C6	—	ClearPath(Ambulance)
R3	C2, R2	{y/Ambulance}	GreenSignal(Ambulance)
R4	C4, C7	{x/CarA}	¬Behind(CarA,y) ∨ ¬GreenSignal(y) ∨ Proceed(CarA)
R5	R4, C8	{y/Ambulance}	¬GreenSignal(Ambulance) ∨ Proceed(CarA)
R6	R5, R3	—	Proceed(CarA)
R7	R6, C9	—	**EMPTY CLAUSE (NIL)**

Because resolving the KB together with the negated goal ¬Proceed(CarA) derives the empty clause, the clause set {KB ∪ ¬Proceed(CarA)} is unsatisfiable. By the **resolution refutation principle**, this proves KB |= Proceed(CarA); i.e. **Proceed(CarA) is logically entailed and TRUE**. ■

(c) Sufficiency Analysis for Two Simultaneous Emergency Vehicles

The current KB (Rules 1–3) is **not sufficient** for two simultaneous emergency vehicles. With the present rules, if two vehicles both satisfy EmergencyType(x), Rule 1 would independently assert ClearPath(x) and hence GreenSignal(x) for both — which can produce a logically consistent but physically unsafe outcome (two conflicting green signals at an intersection, or no priority ordering between the two emergency vehicles).

Gaps identified and required additions:

- **Priority ordering rule:** ∀x∀y: EmergencyType(x) ∧ EmergencyType(y) ∧ HigherPriority(x,y) → ClearPath(x) ∧ ¬ClearPath(y) — needed to arbitrate between two emergency vehicles (e.g. by response urgency or arrival time).
- **Conflict-detection predicate:** Conflicts(x,y) to flag when two vehicles' paths intersect, so the controller does not grant simultaneous ClearPath to both without resolving the conflict first.
- **Temporal / mutual-exclusion facts:** a time-stamped or FIFO ordering fact (e.g. ArrivedAt(x,t)) so that ties are broken deterministically and consistently.
- **Extended Rule 2:** GreenSignal(x) should additionally require ¬(∃y: y≠x ∧ ClearPath(y) ∧ Conflicts(x,y)) to prevent two simultaneously cleared, conflicting paths.

Justification: First-Order Logic guarantees a valid conclusion only from what is explicitly asserted; since the KB has no notion of priority or conflict between two emergency vehicles, it cannot logically derive which vehicle should yield. Adding the rules above restores *completeness* for the two-vehicle scenario while preserving the soundness of the existing resolution proof in part (b).

Case Study 3 — Agricultural Expert Reasoning System (10 Marks)

(a) CNF Conversion — Step by Step

Symbols: SD = SoilDry, IN = IrrigationNeeded, CW = CropWheat, AD = ApplyDripMethod, CR = CropAtRisk.

P1: SoilDry → IrrigationNeeded

```
Eliminate implication (A→B ≡ ¬A∨B): ¬SD ∨ IN
Already a single clause in CNF: (¬SD ∨ IN)
```

P2: (IrrigationNeeded ∧ CropWheat) → ApplyDripMethod

```
Eliminate implication: ¬(IN ∧ CW) ∨ AD
Push ¬ inward (De Morgan): (¬IN ∨ ¬CW) ∨ AD
Flatten disjunction (already CNF): (¬IN ∨ ¬CW ∨ AD)
```

P3: (¬ApplyDripMethod ∧ CropWheat) → CropAtRisk

```
Eliminate implication: ¬(¬AD ∧ CW) ∨ CR
Push ¬ inward (De Morgan): (AD ∨ ¬CW) ∨ CR
Flatten disjunction (already CNF): (AD ∨ ¬CW ∨ CR)
```

Facts (unit clauses):

```
SoilDry = True → (SD)
CropWheat = True → (CW)
```

(b) Resolution Refutation — Prove 'ApplyDripMethod' (AD)

Negate the goal and add ¬AD to the CNF clause set, then resolve to derive the empty clause ■.

#	Resolve	Resolvent
R1	(¬SD ∨ IN) with (SD)	(IN)
R2	(¬IN ∨ ¬CW ∨ AD) with (IN) [R1]	(¬CW ∨ AD)
R3	(¬CW ∨ AD) [R2] with (CW)	(AD)
R4	(AD) [R3] with (¬AD) [negated goal]	**EMPTY CLAUSE ■**

Resolution proof tree:

Level	Clauses combined	Result
1	(SD) + (¬SD ∨ IN)	(IN)
2	(IN) + (¬IN ∨ ¬CW ∨ AD)	(¬CW ∨ AD)
3	(CW) + (¬CW ∨ AD)	(AD)
4	(AD) + (¬AD)	■ (contradiction)

Since the negated goal, together with the KB, resolves to the empty clause, the KB logically entails the goal: **KB |= ApplyDripMethod**. Theorem proven. ■

(c) Effect of Removing the Fact 'SoilDry'

If SoilDry is removed, the clause set reduces to: (¬SD ∨ IN), (¬IN ∨ ¬CW ∨ AD), (AD ∨ ¬CW ∨ CR), (CW). No clause now asserts SD, so R1 (deriving IN) can never fire, and consequently R2 (deriving AD) can never fire either.

Re-running resolution refutation on 'ApplyDripMethod' with this reduced KB fails to reach the empty clause — **ApplyDripMethod is no longer provable** (it becomes logically *undetermined*, not false, under standard open-world / monotonic first-order and propositional semantics).

Does CropAtRisk still follow? P3 requires ¬ApplyDripMethod ∧ CropWheat. CropWheat is still true, but ¬ApplyDripMethod cannot be proven from the KB (AD is merely unprovable, which is logically distinct from ¬AD being provable). Therefore, under sound monotonic inference, **CropAtRisk also cannot be derived**.

The only way to conclude CropAtRisk in this reduced KB is to invoke a non-monotonic reasoning pattern such as the **Closed-World Assumption (CWA)** or **negation-as-failure**: explicitly treating 'AD not provable' as '¬AD holds,' which then satisfies P3's antecedent and fires CropAtRisk. This is a deliberate modelling choice, not a guarantee of classical logical entailment, and should be flagged to the domain expert before being adopted in a safety-relevant agricultural advisory system.

Case Study 4 — Wumpus World Knowledge Agent (10 Marks)

(a) Belief State Construction via Modus Ponens

Given percepts Stench[1,2], Breeze[1,1], Glitter[2,2], apply Modus Ponens (P, P→Q |= Q) to each matching rule:

#	Premise (percept)	Rule Applied	Derived Fact
MP1	Stench[1,2]	Stench[1,2] → WumpusAdjacent[1,2]	WumpusAdjacent[1,2]
MP2	Breeze[1,1]	Breeze[1,1] → PitAdjacent[1,1]	PitAdjacent[1,1]
MP3	Glitter[2,2]	Glitter[x,y] → Gold[x,y], {x/2,y/2}	Gold[2,2]

Derived belief-state facts: { WumpusAdjacent[1,2], PitAdjacent[1,1], Gold[2,2] }

(b) Forward Chaining to Localize the Wumpus and the Pit

Step	Reasoning
1	WumpusAdjacent[1,2] implies the Wumpus occupies one of [1,2]'s neighbours: {[1,1], [1,3], [2,2]}.
2	No Stench is reported from [2,2]'s own neighbourhood evidence (only Glitter), so [2,2] is not corroborated as the Wumpus cell; candidate set narrows to {[1,1], [1,3]}.
3	The agent is standing at [1,1] and has not perceived Stench there, so by rule 1 (contrapositive) the Wumpus is NOT at [1,1]. This eliminates [1,1], leaving [1,3] as the best current estimate for the Wumpus.
4	PitAdjacent[1,1] implies a Pit occupies one of [1,1]'s neighbours: {[1,2], [2,1]}. Since the agent has safely stood at / moved through [1,2] with only a Stench (no Breeze) reported there, [1,2] is not a Pit; therefore the Pit is most likely at [2,1].

Conclusion: Wumpus ≈ [1,3] (best current estimate); Pit ≈ [2,1] (best current estimate) — both remain provisional until confirmed by additional percepts from neighbouring cells.

(c) Safety Evaluation of Path [1,1] → [1,2] → [2,2]

Cell	Percepts	Safety Reasoning	Safe?
[1,1]	Breeze	Start cell; agent is alive and already standing there.	Yes
[1,2]	Stench (no Breeze)	No Breeze here means no Pit is adjacent to [1,2]. Stench indicates Wumpus-adjacency only, and by (b)-Step 3 the Wumpus is not co-located at [1,2] itself.	Yes
[2,2]	Glitter (no Stench/Breeze)	By Rule 4 (¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]), the absence of both Stench and Breeze at [2,2] confirms no adjacent Wumpus or Pit threat here.	Yes

Final Verdict: The path [1,1] → [1,2] → [2,2] is judged **SAFE** for the agent to traverse and collect the Gold at [2,2], since Rule 4 is satisfied at every cell along the route and no cell is corroborated as containing (or being immediately adjacent to an unresolved) Pit or Wumpus threat on the travelled path itself. The agent should nonetheless continue to gather percepts at [1,3] and [2,1] before exploring beyond this route, since the Wumpus/Pit locations there remain only provisionally resolved.

This solution set was independently derived and cross-verified using the accompanying Python inference engine (inference_engine.py), which programmatically reproduces every forward-chaining, backward-chaining, unification, and resolution step shown above. See output.png for the corresponding visual trace and Report.pdf for the consolidated analysis and rubric-based self-evaluation.